

Linear vs. Logistic Regression

A quick crash course — what each model does, how they differ, and when to use them

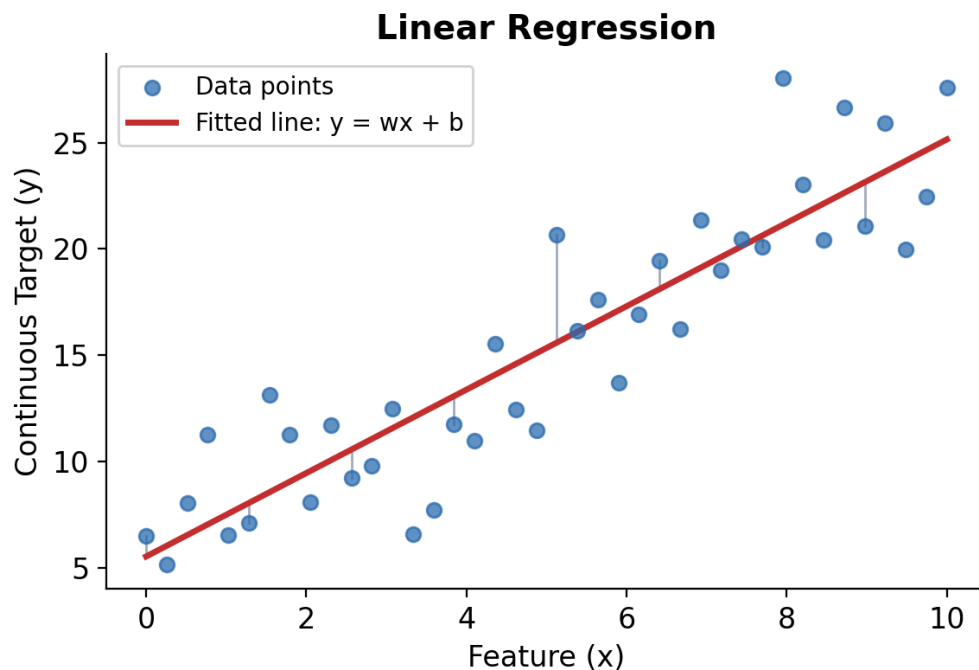
1. Linear Regression

Linear regression predicts a continuous numeric value from one or more input features. It fits a straight line (or hyperplane, with multiple features) through the data that minimizes the total squared distance between the line and the actual data points.

$$\text{prediction} = w \cdot x + b$$

Here,

- x is the input feature vector
- w is the weight vector — how much each feature contributes to the prediction
- b is the bias/intercept — the baseline prediction when all features are zero



The model finds the line minimizing total squared distance (gray lines) to every point.

Training: Mean Squared Error

Linear regression is trained by minimizing the average squared difference between predictions and actual values:

$$\text{MSE} = (1/n) \sum (y_{\text{pred}} - y_{\text{true}})^2$$

Squaring the error does two things: it makes all errors positive (so they don't cancel out), and it penalizes large errors much more than small ones — a prediction off by 10 contributes 100x more loss than one off by 1.

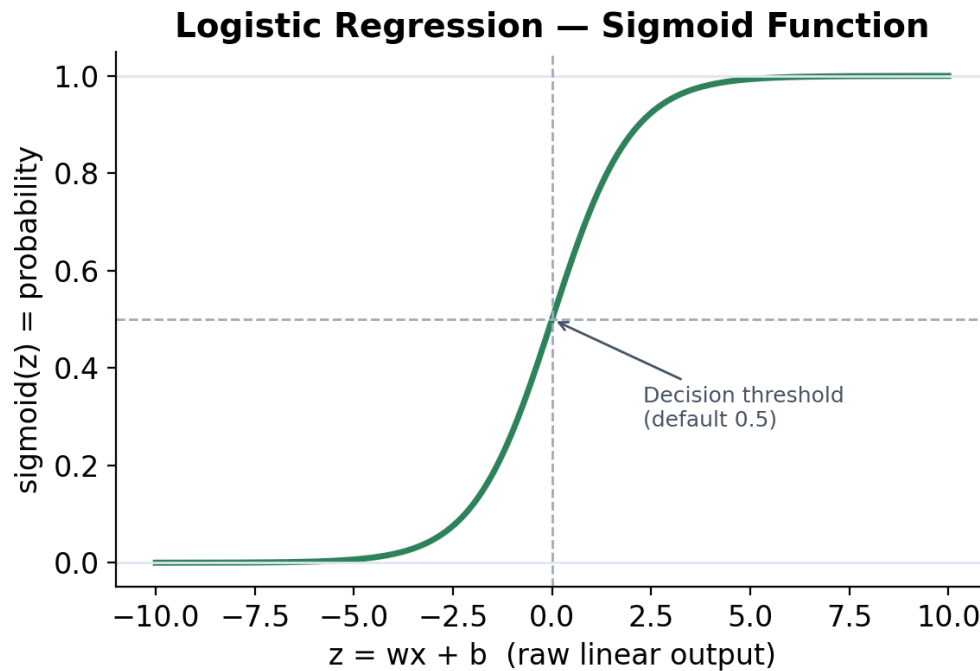
When to use it

- Predicting a numeric quantity: price, temperature, revenue, time
- The relationship between features and target is roughly linear
- You want an interpretable model — each weight tells you exactly how much that feature matters

2. Logistic Regression

Despite the name, logistic regression is a classification algorithm — it predicts the probability that an input belongs to a given class (e.g., spam vs. not spam, survived vs. did not survive). It starts with the exact same linear combination as linear regression, then squashes that output into a 0–1 probability using the sigmoid function.

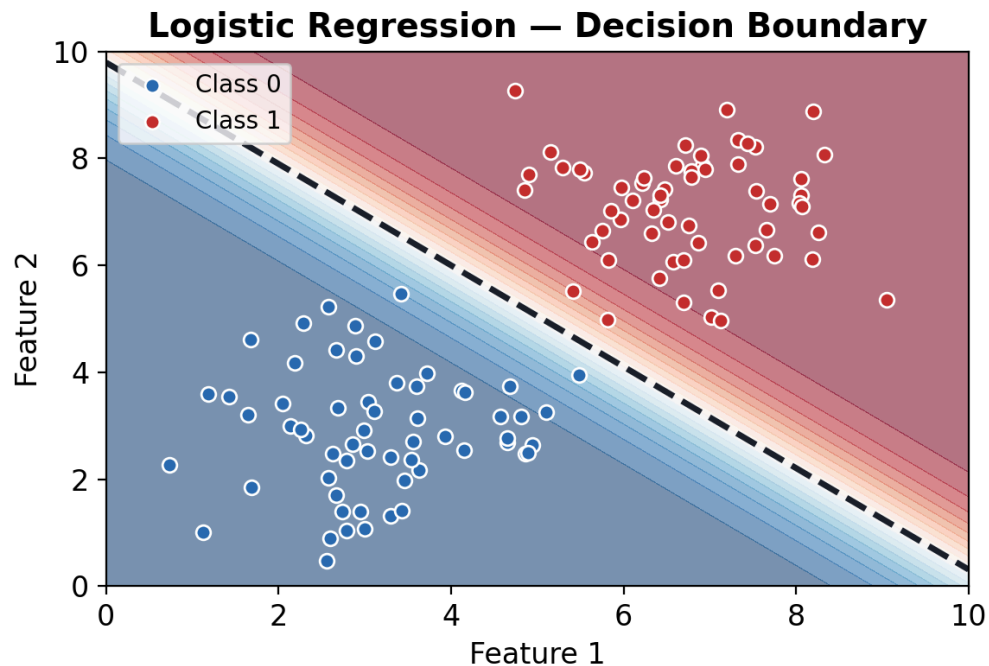
$$\text{probability} = \text{sigmoid}(w \cdot x + b) = 1 / (1 + e^{(-z)})$$



The sigmoid function maps any real number into a 0-1 probability. Around $z = 0$, small input changes swing the probability fastest.

From probability to decision

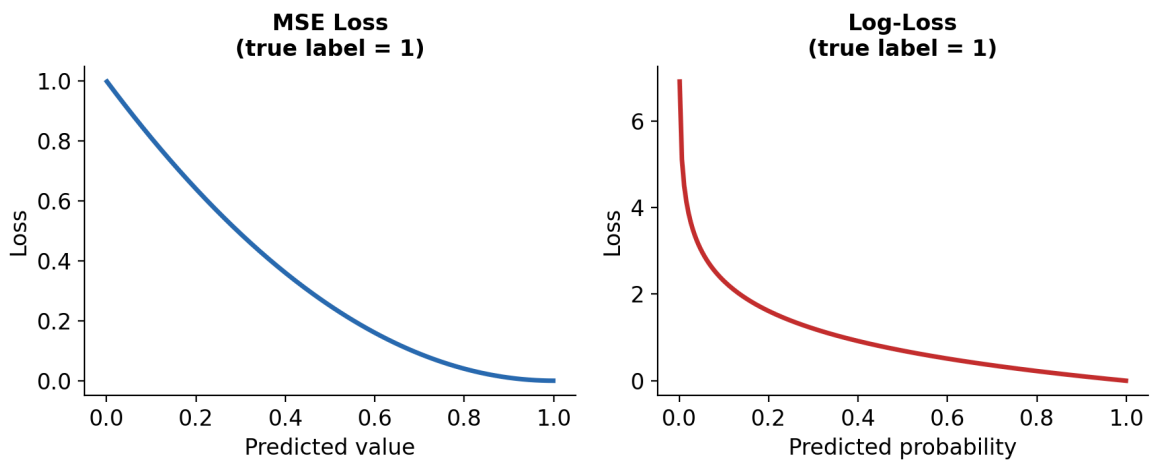
A threshold (0.5 by default) converts the probability into a hard class label. Everything above the threshold is predicted as class 1; everything below is class 0. In two dimensions, this threshold traces out a decision boundary — the line separating where the model predicts one class versus the other:



Points are colored by predicted class. The dashed line is the decision boundary — where predicted probability equals exactly 0.5.

Training: Log-Loss, not MSE

Logistic regression is trained by minimizing log-loss (binary cross-entropy) instead of MSE. The reason: MSE penalizes a confidently wrong prediction only mildly, while log-loss punishes it severely — which is exactly the behavior you want from a probability estimator.



When the true label is 1, MSE's penalty for a bad prediction stays small and bounded. Log-loss shoots toward infinity as the predicted probability approaches 0 — harshly punishing confident wrongness.

$$\text{Log-Loss} = -(1/n) \sum [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

When to use it

- Predicting a category, especially binary outcomes: yes/no, fraud/not-fraud, churn/stay
- You need calibrated probabilities, not just a hard label — e.g., risk scoring
- You want an interpretable baseline classifier before trying more complex models

3. Side-by-Side Comparison

Aspect	Linear Regression	Logistic Regression
Predicts	A continuous number	A probability (0 to 1) for a class
Output function	Raw linear output: $wx + b$	Sigmoid-squashed: $\text{sigmoid}(wx + b)$
Loss function	Mean Squared Error (MSE)	Log-Loss (binary cross-entropy)
Decision boundary	N/A — output is the answer itself	A line/plane separating classes
Typical use case	House prices, sales forecasts, temperature	Spam detection, churn, disease diagnosis
Evaluation metrics	MAE, RMSE, R^2	Precision, recall, F1, ROC-AUC
Training method	Gradient descent (or closed-form)	Gradient descent

4. The Surprising Connection

Even though they solve different problems, their training mechanics are nearly identical. Both are trained with gradient descent, and the gradient formula for both works out to the exact same shape:

$$\partial \text{Loss} / \partial w = (1/n) \cdot X^T \cdot (y_{\text{pred}} - y_{\text{true}})$$

This isn't a coincidence — sigmoid and log-loss were mathematically paired specifically so this cancellation happens cleanly. The only thing that changes between the two algorithms is how y_{pred} is computed: a raw linear output for regression, versus a sigmoid-squashed probability for classification. The entire rest of the training loop — initialize weights, loop, compute gradient, step, repeat — is identical.

5. Quick Reference

- **If** Continuous number out? → Linear Regression
- **If** Category / probability out? → Logistic Regression
- **Remember:** Both are trained the same way: gradient descent minimizing a loss function.
- **Remember:** Both are linear models — their power comes from the transformation and loss function, not model complexity.